

Pull the Trigger

Send a Real Reply

Close the loop — from draft to sent.

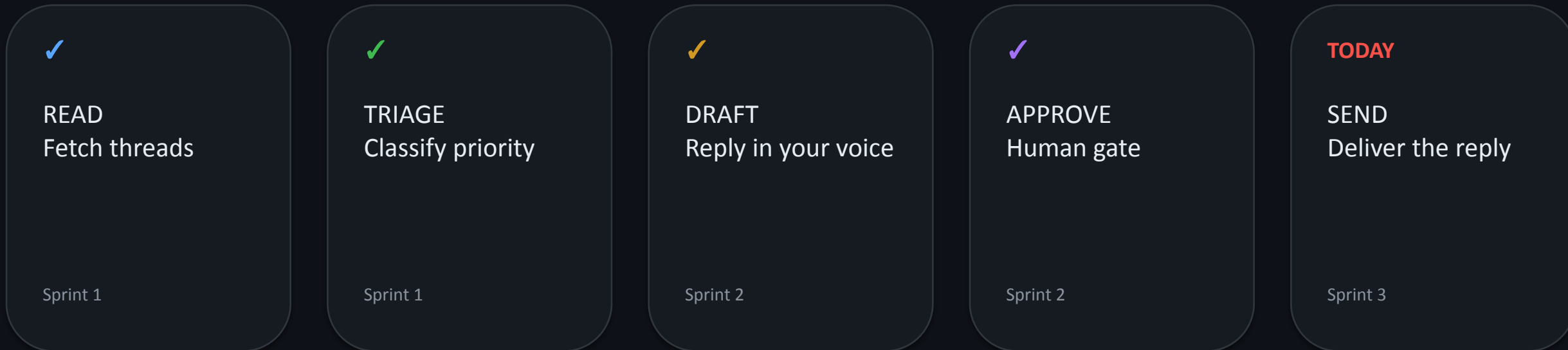
MS 3.1
Skill Sprint

Kiro + Gmail
Free Tools

All Platforms
OS

The Story So Far

We built an agent that reads, triages, and drafts. But it never sends.



Today we close this loop. ↑

What We're Building Today

Integrate send capability directly into our project code.

send_reply()

New function in engine.py — sends email via Gmail API

send_approved_drafts()

Reads approved_drafts.json, sends each unsent draft

approval_gate.py update

Now saves thread_id + recipient_email on approve

CLI: python engine.py send

One command to send all approved drafts

Anatomy of a Threaded Reply

Three things Gmail needs to keep the reply in the same conversation.

1

threadId

Tells Gmail which conversation this belongs to

We already capture this in engine.py

2

In-Reply-To header

Points to the Message-ID of the email you're replying to

Optional but improves threading

3

Subject: "Re: ..."

Gmail uses this as a visual threading cue

Prefix original subject with Re:

The send_reply() Function

What we're adding to engine.py — Kiro will help us write this.

```
def send_reply(thread_id, to, subject, body, message_id=None):
```

```
    service = _build_gmail_service()
```

```
    msg = MIMEText(body, "plain", "utf-8")
```

```
    msg["To"] = to
```

```
    msg["Subject"] = "Re: " + subject # threading cue
```

```
    if message_id:                # threading header
```

```
        msg["In-Reply-To"] = message_id
```

```
    raw = base64.urlsafe_b64encode(msg.as_bytes()).decode()
```

Same `_build_gmail_service()` from Sprint 1 — just needs `gmail.send` scope added.

```
    sent = service.users().messages().send(
```

```
        user_id="me", body={"raw": raw, "threadId": thread_id}
```

Live Demo — Kiro Writes the Code

We prompt Kiro to build `send_reply()` — then we run it ourselves.

Prompt to Kiro:

"Add a `send_reply()` function to `engine.py` that sends an email as a reply to an existing Gmail thread. It should use `MIMEText`, set threading headers, `base64url-encode`, and call `messages.send()` with the `threadId`."

Then we test it:

```
$ python test_send.py
```

✓ Sent! Message ID: 18a3f...
Check your inbox.

Kiro writes the code → You run it → Email arrives in your inbox

Safety First

Always test on yourself before sending to anyone else.

1. Send to YOUR OWN email first

2. Check it arrives in the correct thread

3. Verify formatting, signature, no weird characters

4. Then — and only then — try a real recipient



Once sent, you CANNOT unsend. The approval gate is your last chance to edit.

The Bridge

Connecting the Approval Gate to the Send step.

Approve draft → Click Send → send_reply() → Gmail API → Email delivered!

```
# In app.py — Approval Gate phase:
```

```
if st.button("📧 Send"):  
    result = send_reply(  
        thread_id=thread_id,  
        to=recipient,  
        subject=subject,  
        body=approved_draft
```

```
)
```

One button. Same app. No separate scripts or CLIs — everything in The Draft Desk.
st.success(f"Sent! ID: {result['message_id']}")

What You Did Today



Added gmail.send scope to engine.py (one-line change)



Built send_reply() — sends threaded replies via Gmail API



Added Send button to app.py's Approval Gate



Tested: Approve → Send → email lands in inbox



Same Streamlit app, now with full send capability

Tomorrow — MS 3.2

Calendar Whisperer

Parse a meeting request.
Find a free slot.
Book the event.

Homework: Enable Google
Calendar API in your
Cloud project.



Screenshot your sent email → post on LinkedIn → #MyAIChiefOfStaff